

GENERALIZATION V.S. MEMORIZATION: TRACING LANGUAGE MODELS’ CAPABILITIES BACK TO PRETRAINING DATA

Xinyi Wang^{1*}, Antonis Antoniadis^{1*}, Yanai Elazar^{2,3}, Alfonso Amayuelas¹, Alon Albalak⁴,
Kexun Zhang⁵, William Yang Wang¹

¹University of California, Santa Barbara, ²Allen Institute for AI,

³University of Washington, ⁴SynthLabs, ⁵Carnegie Mellon University

{xinyi.wang, antonis}@ucsb.edu, william@cs.ucsb.edu

ABSTRACT

The impressive capabilities of large language models (LLMs) have sparked debate over whether these models genuinely generalize to unseen tasks or predominantly rely on memorizing vast amounts of pretraining data. To explore this issue, we introduce an extended concept of memorization, **distributional memorization**, which measures the correlation between the LLM output probabilities and the pretraining data frequency. To effectively capture task-specific pretraining data frequency, we propose a novel **task-gram language model**, which is built by counting the co-occurrence of semantically related n -gram pairs from task inputs and outputs in the pretraining corpus. Using the Pythia models trained on the Pile dataset, we evaluate four distinct tasks: machine translation, factual question answering, world knowledge understanding, and math reasoning. Our findings reveal varying levels of memorization, with the strongest effect observed in factual question answering. Furthermore, while model performance improves across all tasks as LLM size increases, only factual question answering shows an increase in memorization, whereas machine translation and reasoning tasks exhibit greater generalization, producing more novel outputs. This study demonstrates that memorization plays a larger role in simpler, knowledge-intensive tasks, while generalization is the key for harder, reasoning-based tasks, providing a scalable method for analyzing large pretraining corpora in greater depth.¹

1 INTRODUCTION

Large language models (LLMs), such as GPT-4, have achieved remarkable performance across a wide range of tasks, yet the debate persists regarding whether these models are truly generalizing to unseen test cases or merely memorizing their extensive training data (Magar and Schwartz, 2022; Srivastava et al., 2024; Bender et al., 2021; Merrill et al., 2024; Shaib et al., 2024). Previous research has primarily investigated memorization in LLMs through verbatim recall of long segments from the training corpus (Zhang et al., 2023; Jiang et al., 2024; Carlini et al., 2022). However, the exact reproduction of long text is relatively uncommon when examining high-level capabilities such as translation and reasoning, particularly when the task output is short, as with world knowledge questions. To advance this line of inquiry, a more flexible definition of memorization is necessary.

Several works have explored the relationship between memorization and generalization (Feldman, 2020; Feldman and Zhang, 2020; Zhang et al., 2023), often employing *counterfactual memorization*, which measures the difference in model performance when a specific training example is excluded. However, these studies typically use small models and datasets, or subsets of larger datasets, and focus primarily on quantifying how much model behavior depends on memorization. They do not fully examine how different model capabilities emerge from the interplay between memorization and

*denotes equal contribution.

¹Our code is available at: <https://github.com/a-antonidias/llm-corpus-search>

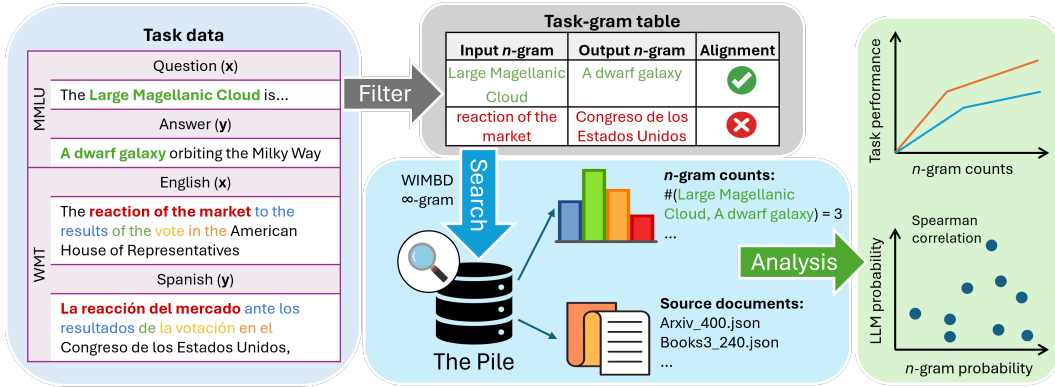


Figure 1: Overview of our proposed analysis pipeline. For the selected evaluation tasks, we first construct a **task-gram table** by matching semantically similar n -grams from task inputs (x) and targets (y). These n -grams are then searched within the pretraining corpus, yielding their counts and source documents. We then build a **task-gram language model** from the obtained counts and then analyze their relationship with LLM predictions.

generalization. Moreover, the limited scale of these analyses is partly constrained by the definition of counterfactual memorization, which requires expensive model retraining.

In this paper, we introduce a new framework for understanding memorization at scale, enabling us to analyze various LLM capabilities. We define **distributional memorization** as the correlation between the distribution of LLM outputs and the distribution of pretraining data. Similarly, we define **distributional generalization** as the divergence between the LLM’s output distribution and the pretraining data distribution. For simplicity, we will refer to these concepts as *memorization* and *generalization* throughout the paper.

Our approach to defining memorization and generalization requires estimating the distribution of LLMs’ pretraining corpora, which is a challenging task given the sheer size of these datasets, often containing trillions of tokens. To address this, we propose a novel method for modeling language distributions by counting semantically related n -gram pairs extracted from a task’s input-output pairs. For example, in machine translation, these n -grams would correspond to phrase pairs from the source and target languages, as shown in Figure 1. This allows us to construct a set of n -gram pairs that characterize the task. Drawing inspiration from the *phrase table* used in machine translation (Passban et al., 2016), which consists of translation pairs, we refer to this set as the task’s **task-gram table**. When n -grams from the input and output co-occur within a document, they are often separated by significant distances. By counting these co-occurrences, we are able to model long-range, task-relevant dependencies (Elazar et al. (2019; 2022)). The resulting n -gram language model, built from the task-gram table, is referred to as a **task-gram language model**. In contrast, classical n -gram LMs capture only local lexical dependencies within a single n -gram. Although Liu et al. (2024) introduced a ∞ -gram LM, which extends n to infinity through backoff from infinitely long n -grams, the effective length after backoff remains limited and cannot capture the long-range dependencies modeled by our task-gram LM. These n -gram LMs can be viewed as empirical approximations of the pretraining data distribution since they follow the observed frequency of n -grams (or pairs) in the data. Using these n -gram LMs, we measure the degree of memorization by correlating LLM-predicted probabilities with the probabilities generated by the n -gram LMs.

Our experiments focus on the Pythia model family (Biderman et al., 2023), pretrained on the Pile dataset (Gao et al., 2020), and evaluate performance across three tasks: translation (WMT (Callison-Burch et al., 2009)), factual question answering (TriviaQA (Joshi et al., 2017)), world knowledge questions (MMLU (Hendrycks et al., 2020)), and math reasoning (GSM8K (Cobbe et al., 2021)). The high-level overview of our analysis pipeline is depicted in Figure 1. We first construct a task-gram table using supervised task data, then search for the co-occurrence of n -gram pairs in the pretraining corpus using WIMBD (Elazar et al., 2024). Finally, we construct a task-gram LM from the co-occurrence counts and compare it with LLM predictions. Our results demonstrate that the task-gram LM effectively captures task-relevant data distributions, providing better explanations for LLM behaviors compared to the ∞ -gram LM (Liu et al., 2024).

Among the four tasks, our analysis reveals that TriviaQA exhibits the strongest memorization effect, with task performance highly correlated to the n -gram distributions in the pretraining data. In contrast, MMLU shows weaker memorization, and WMT exhibits only an insignificant memorization effect. Additionally, our results show that as the model size increases, the source of performance gains varies across tasks: for TriviaQA, improved memorization plays a key role, while for MMLU, WMT, and GSM8K, increased generalization is more crucial. These findings align with the nature of the tasks: TriviaQA relies on factual recall, MMLU and GSM8K require more complex reasoning, and WMT reflects transferable translation skills.

To complement our distributional memorization analysis, we also conduct a gradient-based estimation of the training influence of pretraining documents on test examples. We find that, consistent with our distributional memorization results, pretraining documents have the largest impact on TriviaQA, followed by MMLU, and the least impact on WMT. Documents containing n -gram pairs from the task-gram table exert a greater influence than those containing only individual n -grams.

To our knowledge, this work presents one of the first comprehensive analyses of LLM capabilities by tracing their origins to pretraining corpora at scale. Our task-gram language model offers a scalable, generalizable approach for understanding LLM behavior across a variety of tasks.²

2 METHOD

Diverse abilities have been observed from LLMs trained on large pretraining corpora. Many of them are distinct in nature, like knowledge retrieval and math reasoning. It is reasonable to hypothesize that these capabilities come from different subsets of texts from pretraining. To better identify these task-relevant texts, we propose to construct a **task-gram table** from a set of supervised task data $D_T = \{(x_i, y_i)\}_i$ corresponding to task T , to characterize the task-relevant documents. More specifically, we mine semantically similar n -gram pairs (s^x, s^y) from corresponding task input x and output y respectively. i.e., $s^x \subseteq x$, and $s^y \subseteq y$. We use the cosine similarity between the embeddings of the n -grams to measure the semantic closeness of the n -grams. The task-gram table is then constructed by all possible such n -gram pairs in D_T . The mined n -gram pairs capture task-specific supervision signals as the output n -gram can be viewed as “answering” the input n -gram. We formally define the task-gram table as follows:

Definition 1. Denote all possible combinations of input-output n -gram pairs from task data $D_T = \{(x_i, y_i)\}_i$ by $A_n(T) = \cup_i [G_n(x_i) \times G_n(y_i)]$, where $G_n(\cdot)$ denotes the set of all possible n -grams in a piece of text. Then the **task-gram table** $H_n(T)$ is defined as:

$$H_n(T) = \{(s_j^x, s_j^y) \mid \cos(E(s_j^x), E(s_j^y)) > \gamma_T, s^x \neq s^y, (s_j^x, s_j^y) \in A_n(T)\},$$

where $\gamma_T \in (0, 1)$ is a threshold chosen as a hyperparameter, E is a pretrained text embedding model, and $\cos(\cdot, \cdot)$ denotes the cosine similarity between two vectors.

Based on the task-gram table, we can then construct a **task-gram language model**, which can describe the distribution of the characterized task-related data in a large pretraining corpus. In the following paper, we use C as the counting function. We define the number of co-occurrence of a n -gram pair (s^x, s^y) in the same document in the pretraining corpus $\mathcal{D}$ by $C((s^x, s^y), \mathcal{D})$. We also define the number of occurrences of a n -gram s^y in the pretraining corpus $\mathcal{D}$ by $C(s^y, \mathcal{D})$. Then a task-gram language model can be defined as follows:

Definition 2. A **task-gram language model** is defined over its task-gram table $H_n(T)$. For $\forall (s^x, s^y) \in H_n(T)$ and a corpus of interest $\mathcal{D}$, we define the following probability distribution

$$P_{n, \mathcal{D}}(s^y | s^x) = C((s^x, s^y), \mathcal{D}) / C(s^x, \mathcal{D}). \quad (1)$$

In practice, when we observe that an n -gram s^x exists in an unseen text input, the chance of the corresponding n -gram s^y appearing in the output can be estimated by $P_{n, \mathcal{D}}(s^y | s^x)$.

Suppose we have an LLM pretrained on the corpus $\mathcal{D}$. Considering the zero-shot setting where we prompt the LLM with an instruction text u and an input text x .³ Suppose $s^x \subseteq x$ and $s^y \subseteq y$, we

²Code will be made available upon acceptance of the paper.

³In practice, we use a minimal instruction template to indicate the input and output.